

1. Introduction

Inter-Process Communication (IPC) refers to mechanisms that allow processes to communicate and coordinate with each other. Communication includes sending and receiving data, sharing resources, and synchronizing execution between processes.

IPC is essential because processes in modern operating systems are typically isolated for safety and stability, and therefore need controlled methods to exchange information.

In Linux, IPC can be implemented using several mechanisms:

- *Pipes*
Used for unidirectional (half-duplex) communication between related processes (e.g., parent and child).
 - *Unnamed pipes*: typically created with `pipe`
 - *Named pipes (FIFOs)*: allow communication between unrelated processes
- *Message Queues*
Allow processes to exchange messages in a structured and asynchronous way. Messages are stored in a queue and can be retrieved in order or based on priority
- *Semaphores*
Used for process synchronization rather than data exchange. They help control access to shared resources and prevent race conditions
- *Shared Memory*
The fastest IPC mechanism, where multiple processes share a common *memory segment*. Processes can read and write directly to this memory with synchronization (e.g., using semaphores), that is required to avoid conflicts
- *Sockets*
Provide communication between processes, either on the same machine or over a network. They are commonly used for client-server communication

We have already examined some IPC mechanisms. In this section, we will focus on named semaphores, message queues, and shared memory, followed by an introduction to sockets.

Message queues, semaphores, and shared memory are part of XSI IPC, a standardized set of IPC mechanisms originally introduced in System V UNIX¹.

¹ See the `sysvipc(7)` manual page

2. POSIX Message Queues

Message queues are an IPC mechanism that allow processes or threads to communicate by sending and receiving messages through a queue managed by the operating system. The first implementation of message queues came from System V, but it had several limitations. POSIX message queues were later introduced to provide a simpler API, support message priorities, and offer better integration with modern system features like asynchronous notification.

POSIX message queues are widely used in modern applications. They provide a file-like interface with operations such as `open`, `close`, and `unlink`. Furthermore, they support message priorities directly and integrate seamlessly with I/O multiplexing mechanisms like `select` and `poll` via file descriptors.

Message queues are created and opened using `mq_open`. This function returns a *message queue descriptor* (`mqd_t`), which is used in subsequent operations to refer to the opened queue.

Each message queue is identified by a name of the form `/somename`, which is a null-terminated string of up to `NAME_MAX` (typically 255) characters. The name must begin with a forward slash (`/`) and must not contain any additional slashes. Two processes can communicate using the same queue by passing the same name to `mq_open`.

Messages are sent and received using `mq_send` and `mq_receive`. When a process no longer needs a queue, it closes it using `mq_close`. The queue itself can be removed from the system using `mq_unlink`; when it is no longer required. Queue attributes can be retrieved and, in some cases, modified using `mq_getattr` and `mq_setattr`. Additionally, a process can request asynchronous notification of message arrival on an empty queue using `mq_notify`.

A *message queue descriptor* refers to an *open message queue description*. After a `fork`, the child process inherits copies of its parent's message queue descriptors, and both processes refer to the same underlying open queue. The associated flags (`mq_flags`) are shared between corresponding descriptors.

Each message has an associated *priority*, and messages are delivered to the receiving process in order of highest priority first. Message priorities range from 0 (lowest) to `sysconf(_SC_MQ_PRIO_MAX) - 1` (highest). This value is typically 32768^2 .

Message queues are created in a virtual filesystem that can be mounted using the commands:

```
[linux@ipc] sudo mkdir /dev/mqueue  
[linux@ipc] sudo mount -t mqueue none /dev/mqueue
```

On the mount directory the *sticky bit* is automatically enabled. The message queues on the system can be viewed and manipulated using the commands usually used for files such as `ls` and `rm`. Each file consists in a single line containing information about the queue

```
[linux@ipc] cat /dev/mqueue/mymq  
QSIZE:129 NOTIFY:2 SIGNO:0 NOTIFY_PID:8260
```

2 POSIX only requires support for at least priorities in the range 0 to 31

The fields are:

QSIZE

Number of bytes of data in all messages in the queue

NOTIFY_PID

If this is nonzero, then the process with this PID has used `mq_notify` to register for asynchronous message notification, the remaining fields describe how notification occurs

NOTIFY

Notification method:

0 → `SIGEV_SIGNAL`

1 → `SIGEV_NONE`

2 → `SIGEV_THREAD`

SIGNO

Signal number to be used for `SIGEV_SIGNAL`

Message queues use cases

Message queues can be used in a wide variety of applications, particularly those requiring *asynchronous communication* and coordination between processes.

Print spooler system

In such a system, multiple applications (processes) may request printing services, while a dedicated printer daemon is responsible for managing the physical printing device.

Message queues facilitate communication between these components by organizing the system into producers and a consumer. The applications act as producers, generating print requests, while the printer daemon acts as the consumer, processing those requests.

The workflow proceeds as follows. Applications submit print jobs by sending messages to a message queue. Each message typically contains relevant information such as the file name, the number of copies to be printed, and possibly a priority value encoded in the message type (`mtype`). The printer daemon retrieves messages from the queue and processes them sequentially or according to their priority.

This approach decouples the sending and receiving processes. Applications do not need to wait for the printing operation to complete; instead, they continue execution immediately after submitting their request. The printer daemon handles all jobs asynchronously, improving overall system efficiency.

Task / Job Queues

In a task distribution systems a server can enqueue tasks, such as image processing requests, while multiple worker processes dequeue and execute them. This model resembles modern job processing systems and enables scalable workload distribution.

Logging Systems

In a logging architecture, multiple processes can send log messages to a message queue, while a dedicated logging process retrieves these messages and writes them to disk. This design prevents concurrent write conflicts and centralizes logging functionality.

Client–Server communication on a single machine

Message queues can also be used to implement client–server communication on the same machine. Clients send requests to a queue, and the server process reads and processes them, optionally sending responses back. This provides a lightweight alternative to sockets for local inter-process communication.

2.1 Creating a message queue: `mq_open`, `mq_close`, `mq_unlink`

The library functions are implemented on top of underlying system calls as listed in the following table.³

Library interface	System call
<code>mq_close(3)</code>	<code>close(2)</code>
<code>mq_getattr(3)</code>	<code>mq_getsetattr(2)</code>
<code>mq_notify(3)</code>	<code>mq_notify(2)</code>
<code>mq_open(3)</code>	<code>mq_open(2)</code>
<code>mq_receive(3)</code>	<code>mq_timedreceive(2)</code>
<code>mq_send(3)</code>	<code>mq_timedsend(2)</code>
<code>mq_setattr(3)</code>	<code>mq_getsetattr(2)</code>
<code>mq_timedreceive(3)</code>	<code>mq_timedreceive(2)</code>
<code>mq_timedsend(3)</code>	<code>mq_timedsend(2)</code>
<code>mq_unlink(3)</code>	<code>mq_unlink(2)</code>

Tab 3. POSIX Message queues library functions

To create/open a message queue we can use the `mq_open`⁴ function.

```
#include <fcntl.h>
#include <sys/stat.h>
#include <mqueue.h>

mqd_t mq_open(const char *name, int oflag);

mqd_t mq_open(const char *name, int oflag, mode_t mode, struct mq_attr
              *attr);

Return: a message queue descriptor, on error (mqd_t) -1 and errno is set
```

The `mq_open` function creates a new message queue or opens an existing queue. The queue is identified by `name`.

The `oflag` argument specifies the flags that control the operation as in `open`, and exactly only one flag must be specified.

The `mode` argument, in the case of `O_CREAT`, specifies the permissions for the new queue.

The `attr` argument is a structure defined as follows

³ See the `mq_overview(7)` and `mqueue.h(7POSIX)` manual pages

⁴ See the `mq_open(3)` manual page

```

struct mq_attr {
    long mq_flags;      /* Flags (ignored for mq_open()) */
    long mq_maxmsg;     /* Max. # of messages on queue */
    long mq_msgsize;    /* Max. message size (bytes) */
    long mq_curmsgs;    /* # of messages currently in queue ignored for
                        mq_open() */
};

```

When calling `mq_open`, only `mq_maxmsg` and `mq_msgsize` are employed, the other fields are ignored. If `attr` is `NULL`, then the queue is created with implementation-defined default attributes.

A descriptor returned by `mq_open` has the *close-on-exec* flag automatically set.

To close a message queue we can use `mq_close`⁵.

```

#include <mqueue.h>

int mq_close(mqd_t mqdes);

Returns: 0 on success, -1 on error and errno is set

```

The `mq_close` function closes the message queue descriptor `mqdes`. If the calling process has attached a notification request (`mq_notify`) to this message queue via `mqdes`, then this request is removed.

To remove a message queue we can use `mq_unlink`⁶.

```

#include <mqueue.h>

int mq_unlink(const char *name);

Returns: 0 on success, -1 on error and errno is set

```

The `mq_unlink` function removes the specified message queue name. The message queue name is removed immediately. The queue itself is destroyed once any other processes that have the queue open close their descriptors referring to the queue.

⁵ See the `mq_close(3)` manual page

⁶ See the `mq_unlink(3)` manul page